Lecture 17: Virtualization

Some slides borrowed from:
Jaswinder Pal Singh (Princeton), Harsha V. Madhyastha (Michigan), UC Berkeley CS162, and Branden Ghena (Northwestern),

- **Virtualization**

- Approaches
  - Emulation
  - Hypervisors
  - Containers

Virtualization

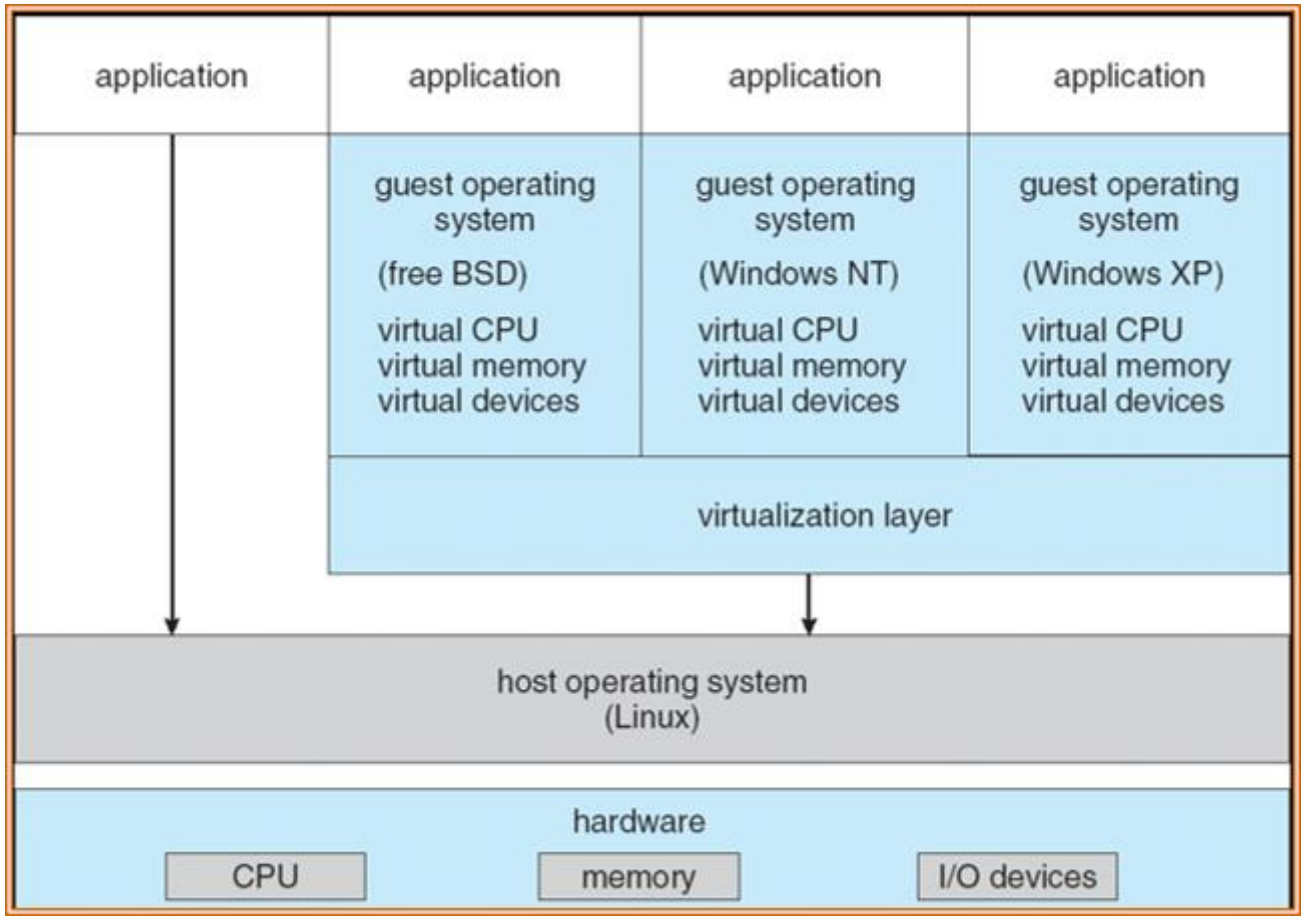
- Virtual versions of real resources are used for protection and limitation
  - Memory – virtual memory
  - CPU – processes and scheduler
  - Disk – file systems

- OS provides these abstractions to simplify applications
  - And provide security

Virtual Machines

- What about virtualizing the whole computer?
  - Provide interfaces that look like a normal computer
  - But actually interact with software that manages access

- Run an entire OS within an OS



Original motivation: support more applications

- 1960s IBM mainframes had many different OSes
  - Some applications only written for certain OSes though

- Virtualization allowed multiple OSes to run on a single mainframe
  - Which let one powerful computer serve varied needs of many people

- Still applies today to some degree
  - Windows + Ubuntu VM
    - Want PowerPoint and also terminal environment (vim/make/gcc)

  - MacOS + Windows VM
    - Various Windows-only programs
    - Often dual boot rather than VM

Modern motivation: package and isolate applications

- High-performance applications aren't really standalone
  - Assumptions about OS
  - Assumptions about libraries and services
  - Multiple processes working together

- A virtual machine is a method to encapsulate "entire stack"
  - Even down to expectations of hardware

- Cloud computing platforms run many applications together
  - Need isolation from each other in a strongly controllable way
    - Exactly 2 GB of RAM go to this
    - Exactly two processor cores go to that

Virtualization approaches

1. Simulate everything in the computer completely
   - Emulation

2. Simulate parts of the computer, but not all of it (really use CPU)
   - Hypervisor

3. Simulate the operating system (software environment)
   - Containers

- Virtualization

- **Approaches**
  - **Emulation**

  - Hypervisors

  - Containers

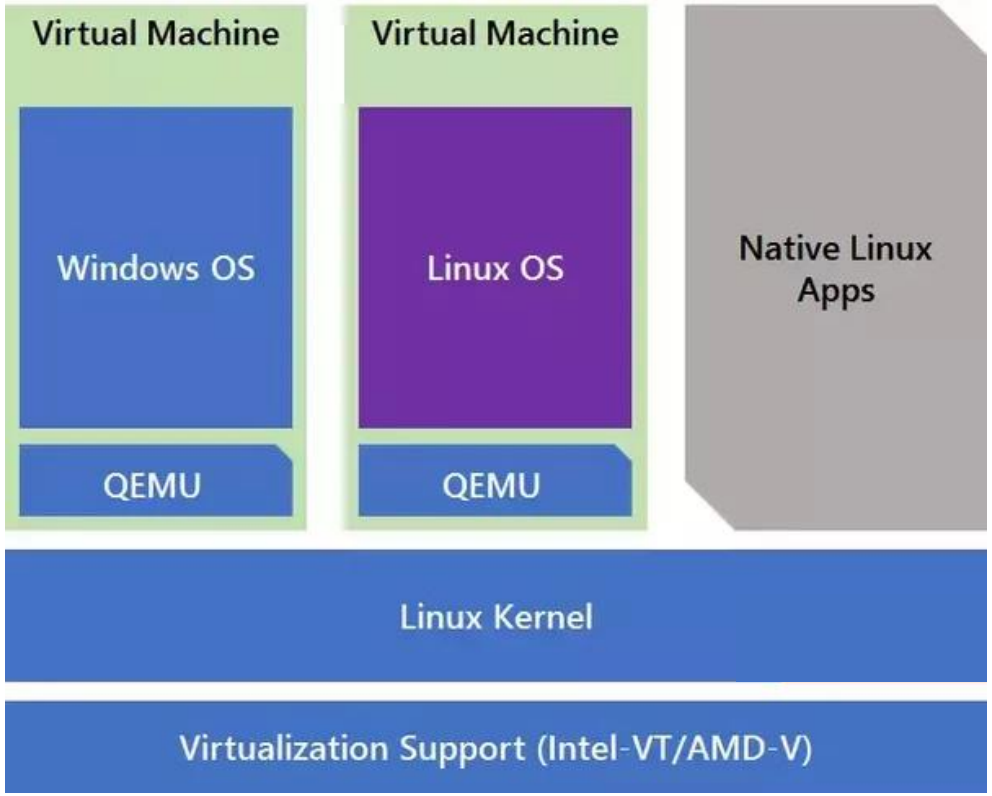
Software emulation

- User software emulates the behavior of every single instruction
  - Data structures for Processor, Memory, I/O, etc.
  - Code for Instruction Cycle:
    - Fetch next instruction
    - Decode
    - Perform operation
    - Update state (e.g., put results into some register but it’s actually the memory)

- Example: Gameboy emulator
  - Simulates every behavior as-if it were actually Gameboy hardware

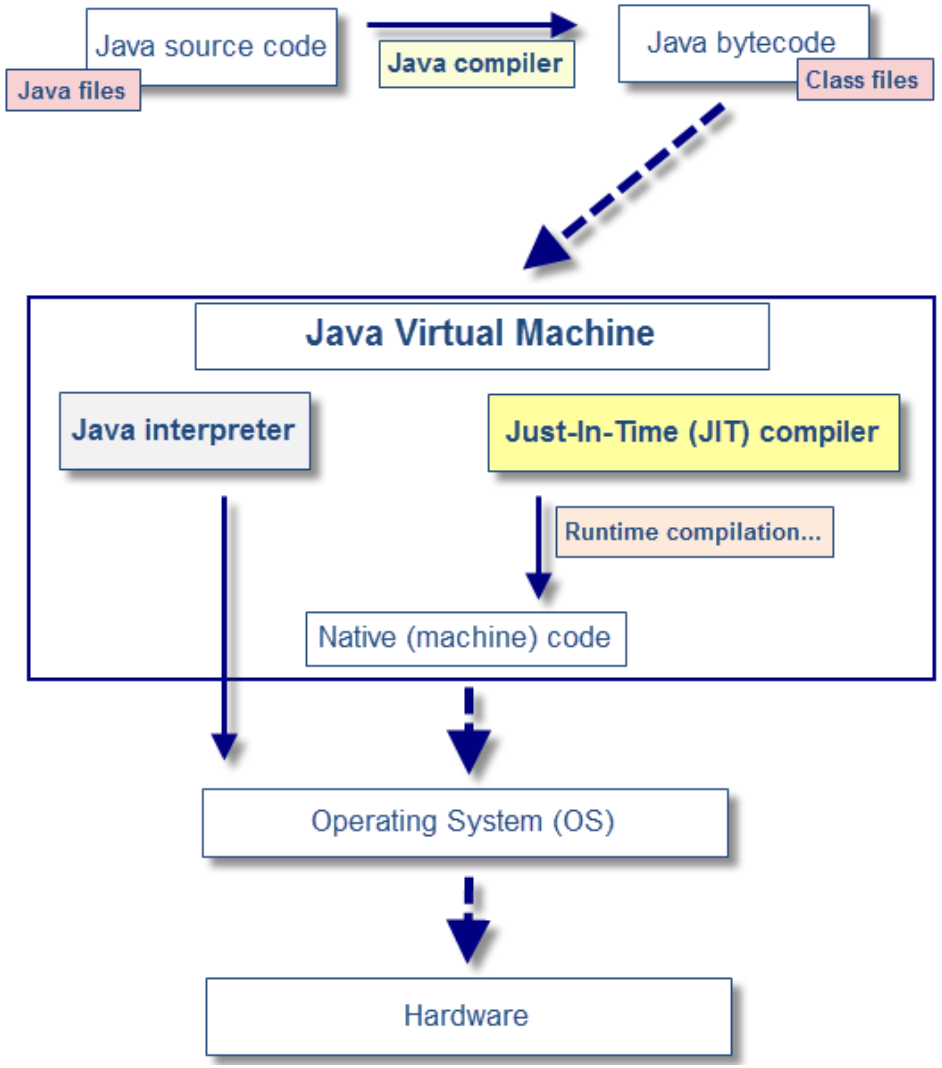
Emulation tradeoffs

- Upsides
  - Any hardware you want
  - Entirely in userspace
- Downside
  - Slower than real hardware by definition



Simple emulators: interpreted languages

- Create a simple environment for code to execute within
- Interpret code instructions (bytecode or lines of code) and perform actions
  - Example: fakes a machine that executes Java bytecode
- Still ties in to many parts of the real machine
  - Filesystem
  - Devices



Not-quite-emulation: binary translation

- MacOS on ARM (M1)
  - Uses ARM processor with ARM instruction set
  - Old programs were compiled for x86-64 instruction set
- Solution: translate assembly instructions
  - Ahead of time (AoT)
  - Or just-in-time (JIT)
  - Works fine for applications that are I/O bound
- Simulates a different CPU, but leaves the remainder of the computer the same

- Virtualization

- **Approaches**
  - Emulation
  - **Hypervisors**
  - Containers

How do we speed up virtual machines?

"Efficiency … demands that a statistically dominant subset of the virtual processor's instructions be executed directly by the real processor, with no software intervention…"

—Popek and Goldberg, 1974

- Need to use some parts of the computer for real while simulating other parts

Virtual Machine Monitor (VMM)

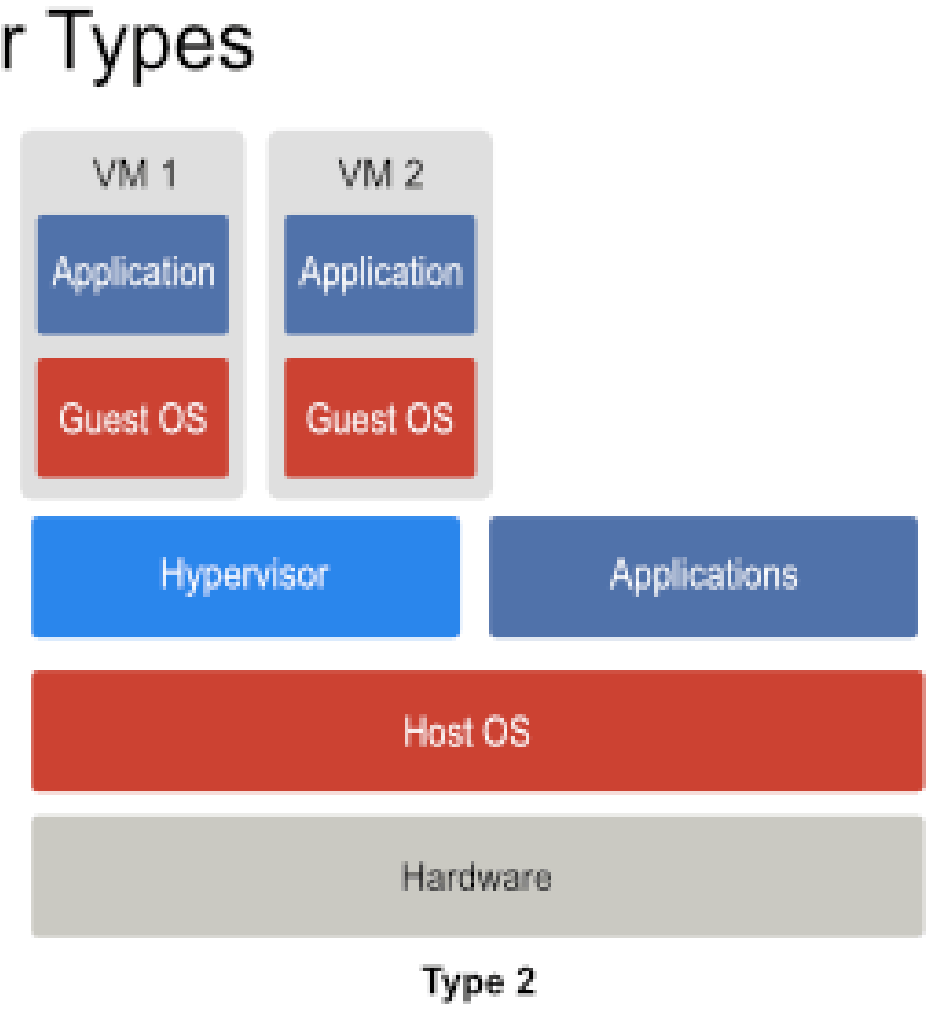
- Also known as hypervisors
  - OS kernel is the system “supervisor” and manages the computer
  - Hypervisor manages supervisors

- Creates the illusion that the OS has full control over the hardware
  - And even gives real (limited) access to hardware whenever possible
  - But may actually be sharing full computer resources among several OSes

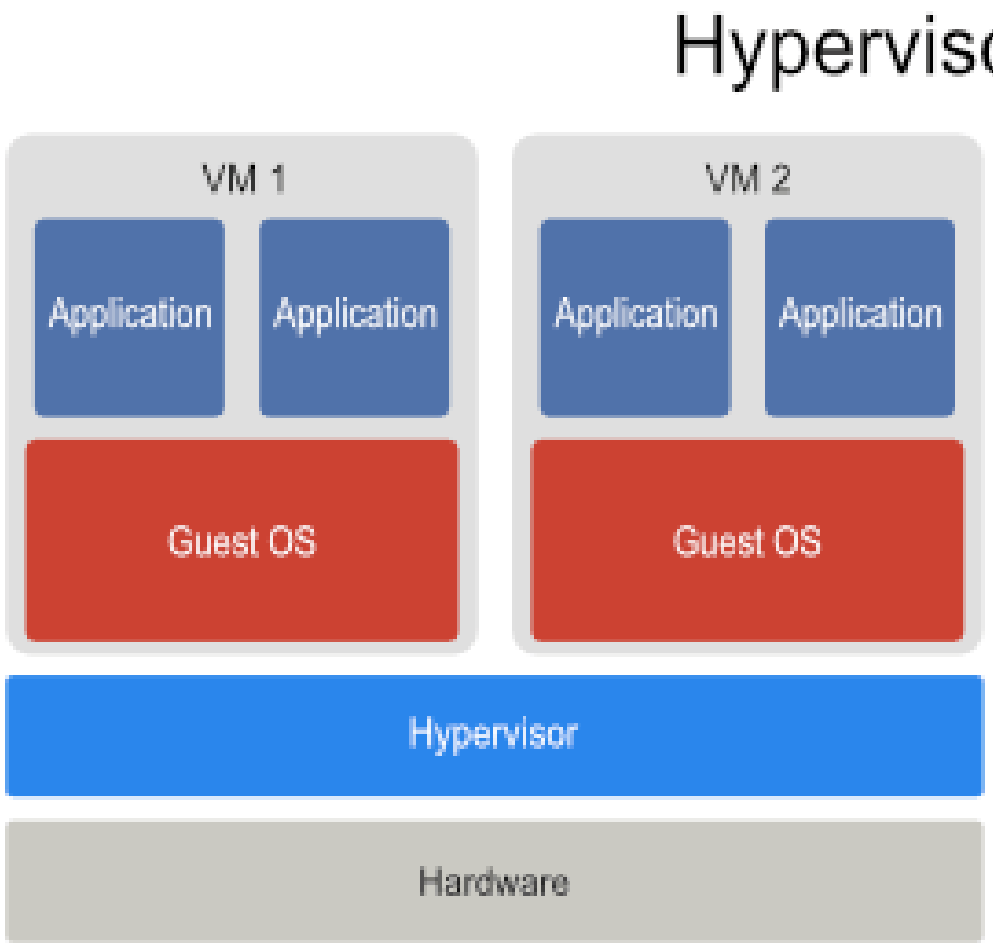
- Probably what you had in mind as virtual machines
  - VirtualBox, VMWare, Parallels

Hypervisor layering

- Normal operating system runs on hardware
  - Known as "Host OS"
- Hypervisor runs on top of host and coordinates with it to enable interactions with hardware
  - Some coordination may be within the kernel itself



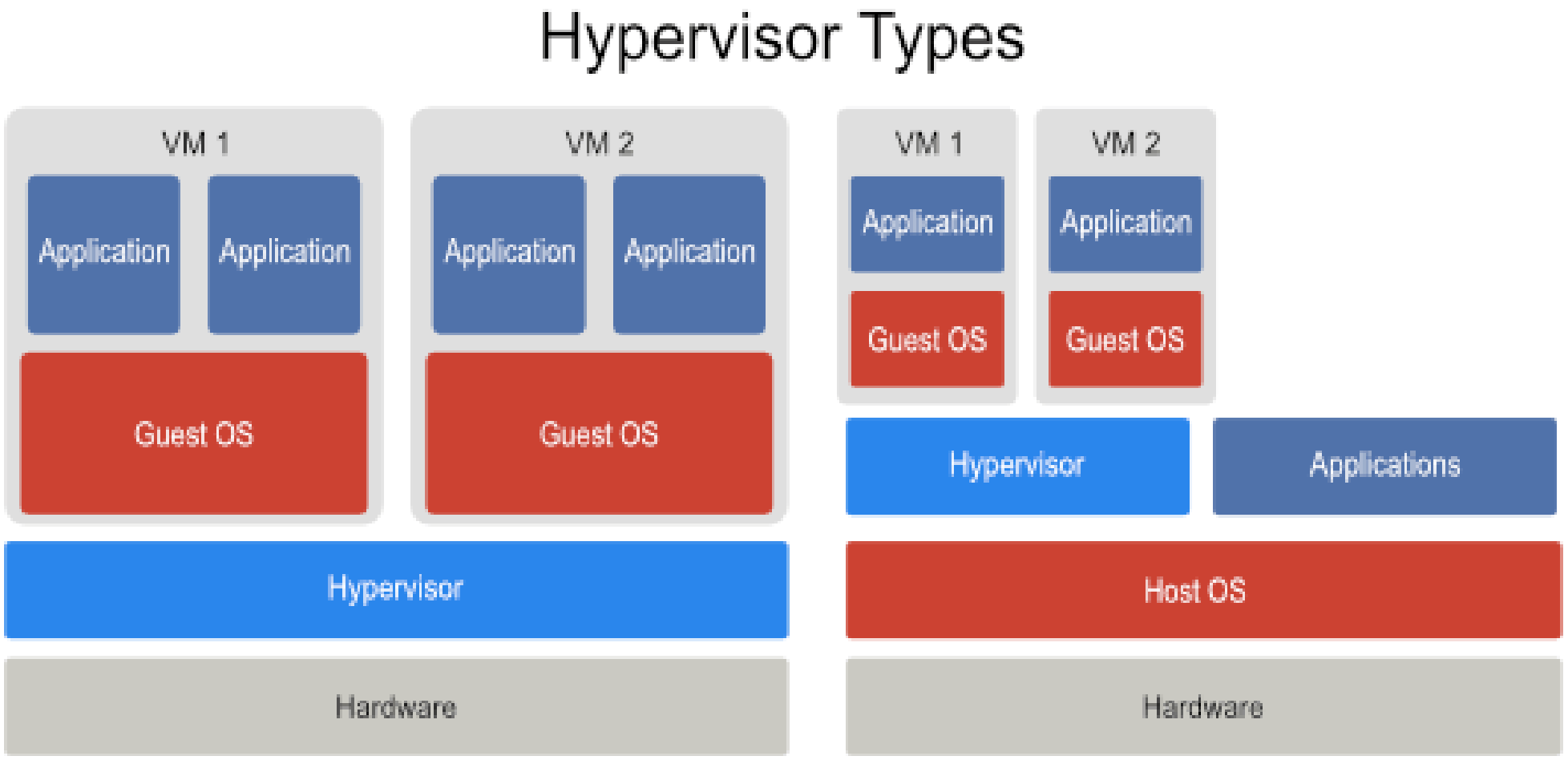
Hypervisor layering



Type 1
"Bare Metal" Hypervisor

- Hypervisor manages hardware directly
- Guest OSes run on top of it

Hypervisor layering



Type 1
"Bare Metal" Hypervisor

Type 2
Hosted Hypervisor

Abstraction choices for hypervisor

- Fully virtualizing hypervisor
  - Virtual machine looks exactly like a physical machine
    - Though not necessarily the same machine it's running on
  - Guest OS does not need to be modified in any way
  - Guest may not even be aware it's running virtually

- Para-virtualizing hypervisor
  - Guest OS has extensions to cooperate with hypervisor
  - Sacrifice transparency for better performance
  - Might include an API to interact with hypervisor
  - Guest OS likely can skip some stuff the hypervisor handles instead

Paravirtualization

- In pure virtualization the VMM intercepts and translates privileged instructions
- “Paravirtualization” means that the guest OS is modified to run on a VMM
- In paravirtualized VM, the guest OS can issue “hypercalls” to the hypervisor
  - Hypercalls are the syscall equivalent. i.e., application:syscall :: kernel:hypercall
- Paravirtualization supported by Windows and Linux

Arbitrary combinations of these are possible

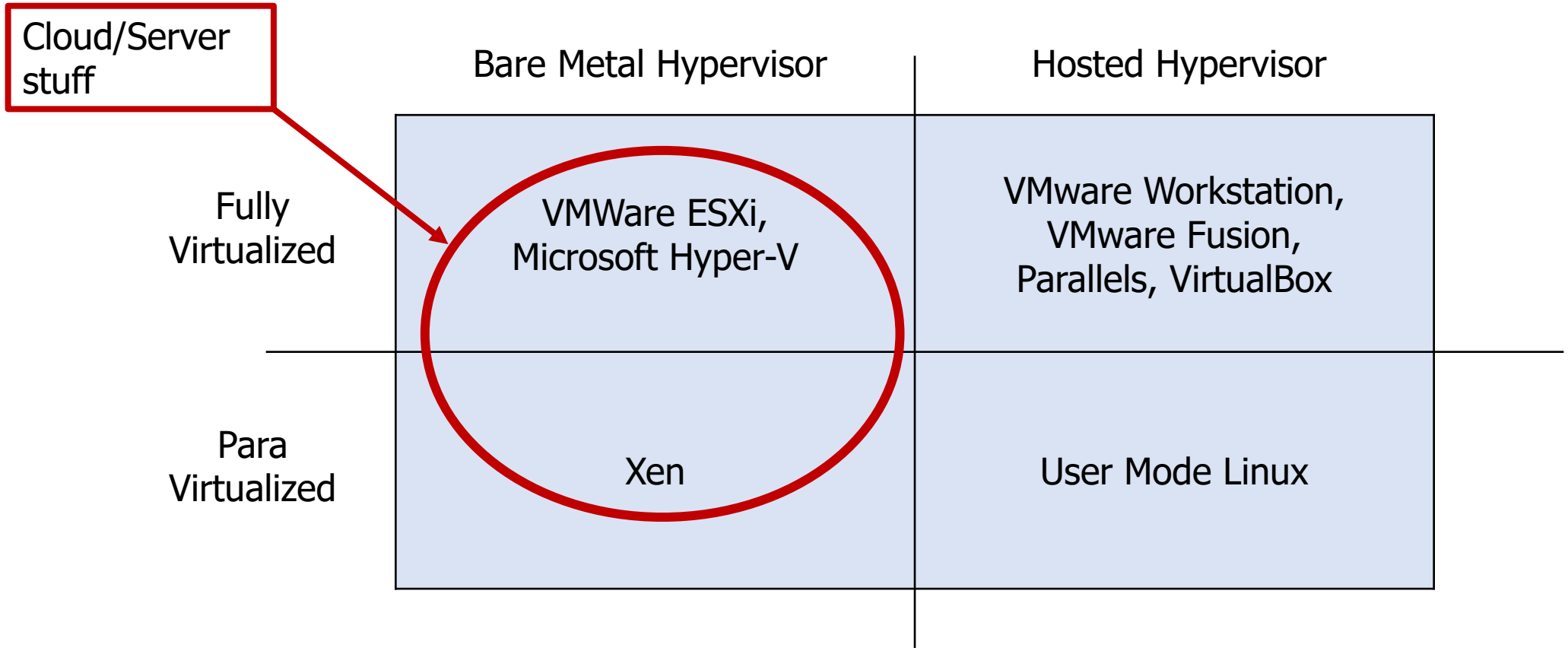
	Bare Metal Hypervisor	Hosted Hypervisor
Fully Virtualized	VMWare ESXi, Microsoft Hyper-V	VMware Workstation, VMware Fusion, Parallels, VirtualBox
Para Virtualized	Xen	User Mode Linux

Arbitrary combinations of these are possible

	Bare Metal Hypervisor	Hosted Hypervisor
Fully Virtualized	VMWare ESXi, Microsoft Hyper-V	VMware Workstation, VMware Fusion, Parallels, VirtualBox
Para Virtualized	Xen	User Mode Linux

"Normal" VMs

Arbitrary combinations of these are possible



Arbitrary combinations of these are possible

	Bare Metal Hypervisor	Hosted Hypervisor
Fully Virtualized	VMWare ESXi, Microsoft Hyper-V	VMware Workstation, VMware Fusion, Parallels, VirtualBox
Para Virtualized	Xen	User Mode Linux

Run Linux as a user-space app on Linux. The guest kernel uses the host kernel underneath. You have to break two kernels, not one.

Hypervisor example: system call

Process	Guest OS	Hypervisor
1. System call: trap to OS		
		2. Receive trap. Call guest OS trap handler
	3. OS trap handler: Decode trap and execute syscall. When done issue return-from-trap	
		4. OS tried to return from trap. Do real return-from-trap
5. Resume execution		

- Virtualization

- **Approaches**
  - Emulation
  - Hypervisors
  - **Containers**

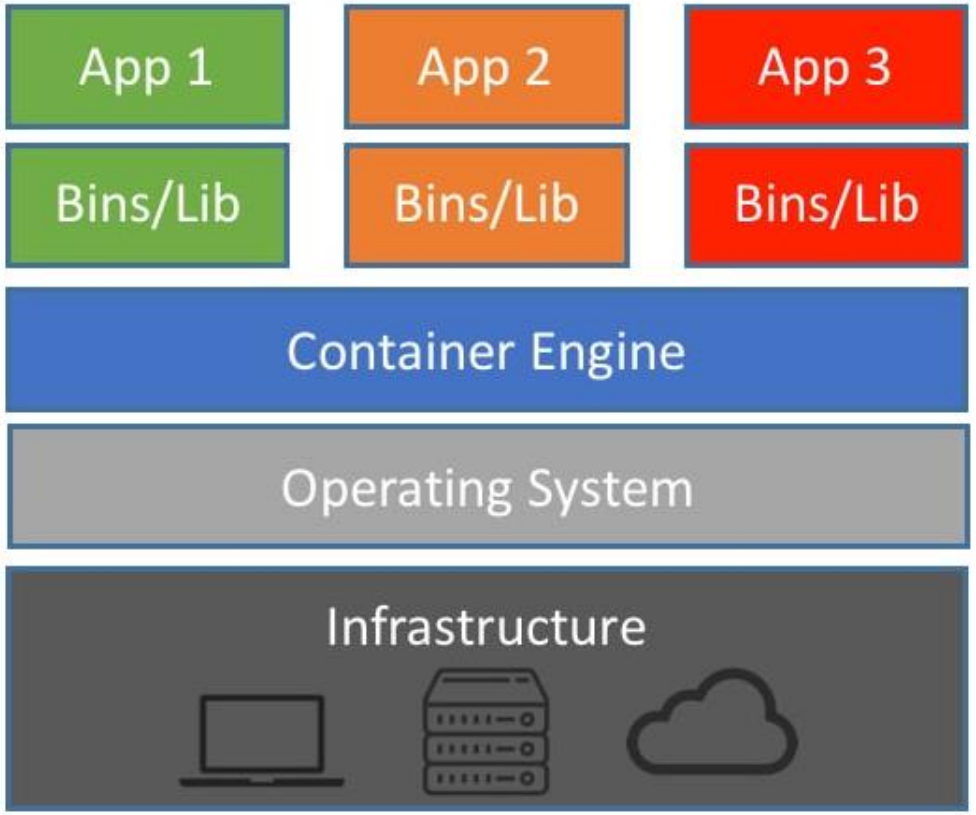
Cloud platform requirements

- May want to provide multiple OSes, but can do so with multiple physical machines

- Really want encapsulation and isolation
  - Encapsulation
    - Include particular shared libraries that application needs
    - Without interfering with other applications on system
  - Isolation
    - Guarantee certain processing and memory allocations to each application
    - Limit visibility into the filesystem (without overhead of partition per app)

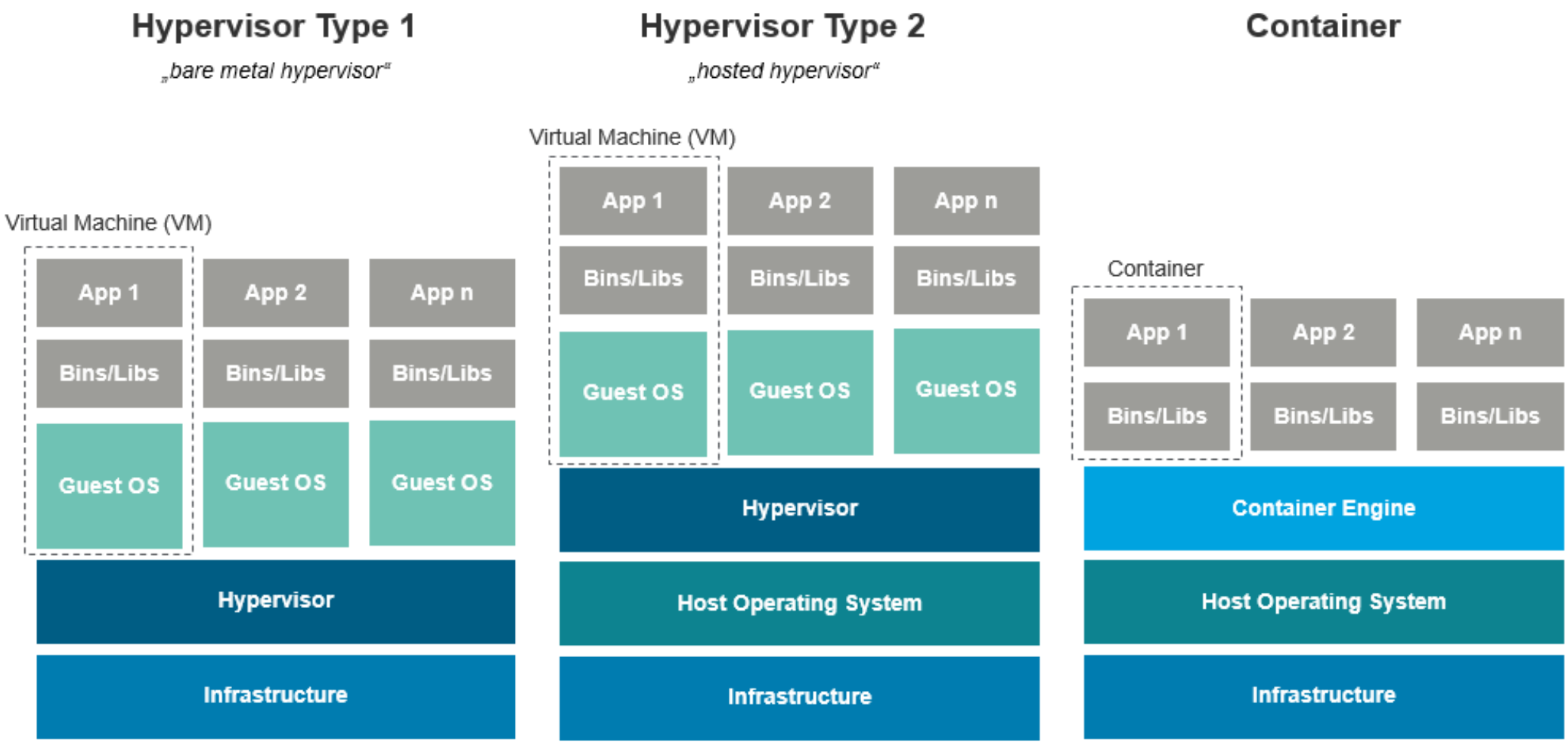
Containers

- Provide each application with illusion of its own dedicated OS
  - Isolated resources: processor and memory
  - Isolated namespace: PIDs, network, filesystem
  - Includes only the binaries and libraries it needs



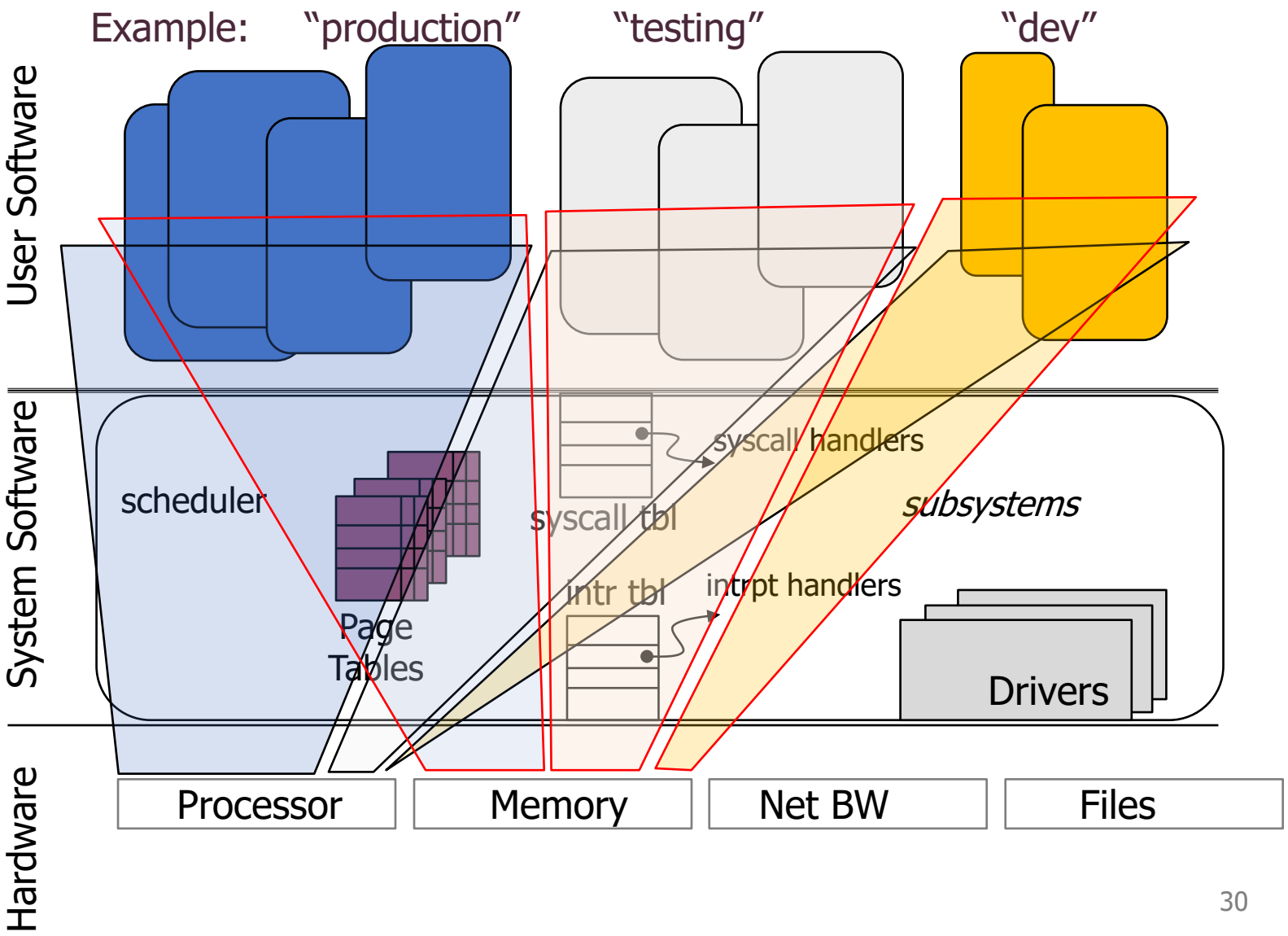
Containers

Visual comparison of Hypervisors and Containers



Linux cgroups (control groups)

- Collection of processes treated as a group for resource allocation
- Provider greater performance isolation between cgroups than between processes
  - Firm limits per group



Cgroups can be used to build containers

- Devices can be connected or denied to a cgroup
  - Cgroup processes will not be able to detect device at all

- Accounting can be done on cgroup usage
  - Memory, CPU, disk I/O

Docker

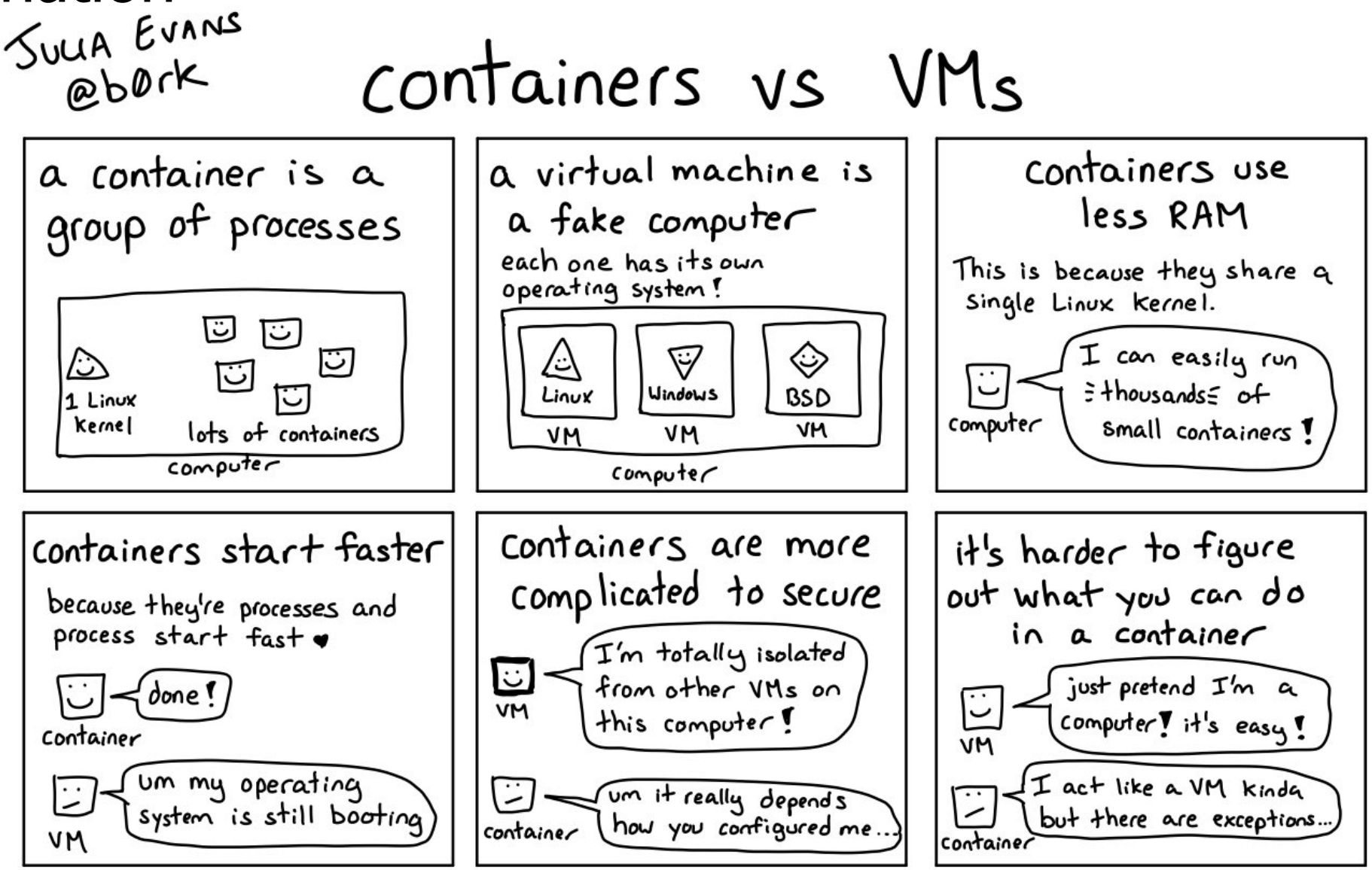
- Container packaging, distribution, and execution
  - Also created open standard for container runtimes
- Images
  - Describes starting state of a Docker container
  - Like a snapshot of the system
- Union file system
  - Image describes file system as a sequence of layers
    - Each layer includes some files
  - Overall file system is the *union* of all the layers
  - Layers can be reused in different images

Docker use cases

- Environments are hard to set up
  - Often the hardest part of starting software development
  - Containerized applications encapsulate requirements
  - Can be run on any system that has the same kernel it was built for

- Packages an application and its requirements into a container
  - Can be used by an individual to more easily run an application
  - Can be deployed to a cloud server to run

Bonus explanation



https://twitter.com/b0rk/status/1237744128450072578

- Virtualization

- Approaches
  - Emulation
  - Hypervisors
  - Containers